

SecureDB Agent: Shielding databases from Agentic AI

Detecting and blocking malicious LLM generated SQL queries

By:

Rajani Nagaraju

Liverpool John Moores University

Masters in Machine Learning & AI

Under the guidance of

Dr. Randheer Bhagi





Introduction

Agentic AI systems, typically driven by Large Language Models (LLMs), operate autonomously - perceiving, reasoning, planning, and executing multi-step actions. In enterprise settings, their unsupervised interaction with databases by generating SQL queries introduces a novel class of operational and security vulnerabilities.

What is Agentic AI?

LLM-driven systems performing autonomous, multi-step actions in database environments.

New Security Paradigm

Unlike human-generated SQL, LLMs pose unique challenges due to NL2SQL conversion without understanding the schema, policy, or data sensitivity

Core Vulnerability

LLMs treat all inputs as an undifferentiated token stream, blurring data and control boundaries, unlike classical SQL injection.

Novel Vulnerabilities and Enterprise Impact

Agentic AI pipelines introduce several new categories of risk, with significant implications for enterprise data security and compliance.

1

Prompt-to-SQL Manipulation

Malicious instructions embedded in user prompts can coerce LLMs into generating unsafe or unintended SQL queries (P2SQL).

2

Model-Level Compromise

Backdoor attacks, such as "ToxicSQL," activate malicious SQL payloads triggered by rare or specific input patterns.

3

Hallucinated/Over-Privileged Queries

LLMs may generate syntactically valid but policy-violating SQL, potentially accessing data beyond authorised user roles.

4

Multi-Stage Agentic Exploits

Complex attack chains involving memory poisoning, tool-output injection, and reasoning-step smuggling compromise agent integrity.

Impact on Enterprise Environments



Data Leakage

Risk of exposure for sensitive customer, financial, or healthcare information.



Privilege Escalation

Agents may gain unauthorised access due to schema-agnostic reasoning capabilities.



Operational Disruption

Potential for expensive queries, infinite loops, or system instability.



Compliance Challenges

Difficulty in maintaining auditability and adhering to regulations (GDPR, HIPAA, PCI-DSS) with autonomous agents.

From Classic SQL Injection to Agentic AI Threats

1

2000s – Classic SQL Injection

- Repeatedly listed in OWASP Top 10 web vulnerabilities.
- Attacks exploit unsanitised user input.
- Bypass authentication, read/modify data, or drop tables.

2

2010s – ML & Deep Learning Defences

- Rule-based WAFs were brittle, missed obfuscated payloads.
- ML classifiers (TF-IDF, n-grams, SVM/RF) achieved $\geq 99.8\%$ accuracy.
- Hybrid CNN/BiLSTM/GRU-attention models achieved $\approx 99\text{--}99.99\%$ accuracy.

3

2023+ – NL2SQL & Agentic AI Era

- LLM-driven Text-to-SQL systems make LLM an active part of query pipeline.
- New LLM-specific risks: Prompt-to-SQL injection, indirect prompt injection.
- ToxicSQL: 0.44% poisoned data can plant persistent SQL backdoors.
- Hallucinated, over-permissive SQL from training code.
- Real-world agent frameworks show prompt injection, tool hijacking, unsafe SQL.

Key Trend

The threat is shifting from purely syntactic SQLi to semantic manipulation of LLMs and their tool calls.

Classical SQLi defences were not designed for this agentic setting.

Existing Defences – Why they fall short for Agentic LLMs

1

Traditional Hardening

- Parameterised queries, input validation, and stored procedures are widely recommended, but security hygiene is often inconsistent in complex, distributed systems. (Kingthorin & Zbraiterman, 2025)
- Static perimeter controls, such as RBAC or ABAC in databases, remain essential but operate on the assumption that the entity issuing SQL is trustworthy. (Gupta et al., 2023)
- Industry blogs still advise isolating LLM Text-to-SQL behind stored procedures instead of allowing direct SQL generation, highlighting ongoing concern. (Eladio Rincón, 2025)

2

ML / DL-based SQLi Detectors

- ML and DL models (SVM, CNN, LSTM, hybrids) achieve near-perfect accuracy on static SQLi benchmarks ($\geq 99-99.99\%$).
- However, these models are trained on fixed attack corpora; they learn pattern signatures, not higher-level malicious intent.
- They struggle with polymorphic payloads and zero-day patterns and rarely incorporate user or schema context. (Dasari et al., 2025; Chen et al., 2021)

3

LLM / Agentic Defences

- New guidance, such as the OWASP Top 10 for LLMs, emphasises prompt hardening, model isolation, and output monitoring for LLM applications. (Kidd & Lange, 2025; Basic & Giaretta, 2025)
- Research on LLM data leakage and Text-to-SQL evaluation mainly focuses on measuring over-exposure, not enforcing runtime access policies. (SecureSQL–Song et al., 2024; Dhinakaran, 2024)
- Existing SQLi detectors are rarely combined with role-aware context or with explainable reasoning about LLM-generated SQL decisions. (Augustine et al., 2024; Lansky et al., 2021)

📌 Identified Research Gap → SecureDBAgent

The defence landscape is fragmented and syntax-centric: static RBAC on one side, pattern-based ML on the other, with no unified view of user role, schema sensitivity, and query semantics.

There is no cohesive, multi-layered, and explainable framework that:

- Enforces context-aware policies at runtime,
- Analyses both structural and semantic features of LLM-generated SQL, and
- Produces human-interpretable justifications for security decisions.

The Imperative for Secure AI-to-Database Interaction

Organisations are rapidly adopting LLM agents for critical operations. However, this introduces new vulnerabilities due to the lack of formal security patterns for AI-to-database interaction.



This approach addresses critical gaps in current security frameworks, which lack specific models for AI-generated malicious SQL and comprehensive explainability in security decisions.



Problem Statement

How can we design a secure, interpretable, and context-aware mediation layer that prevents malicious or unsafe SQL actions generated by agentic AI systems while maintaining seamless operational workflow?

Aims and Objectives

Our primary aim is to mitigate risks in enterprise database systems posed by autonomous AI agents. We will develop a comprehensive framework for safe evaluation of LLM-generated SQL, focusing on intelligent detection, context-aware access control, and transparent, explainable decision-making.

1

Hybrid Detection Engine

Develop an engine combining static (structural) and deep learning (semantic) analysis to classify SQL queries as benign or malicious.

2

Runtime Policy Engine

Implement a context-aware policy engine to enforce access rules based on agent role and database schema, blocking non-compliant queries.

3

Explainability Module

Design a module to provide human-interpretable rationales for blocked queries, enhancing transparency and auditability.

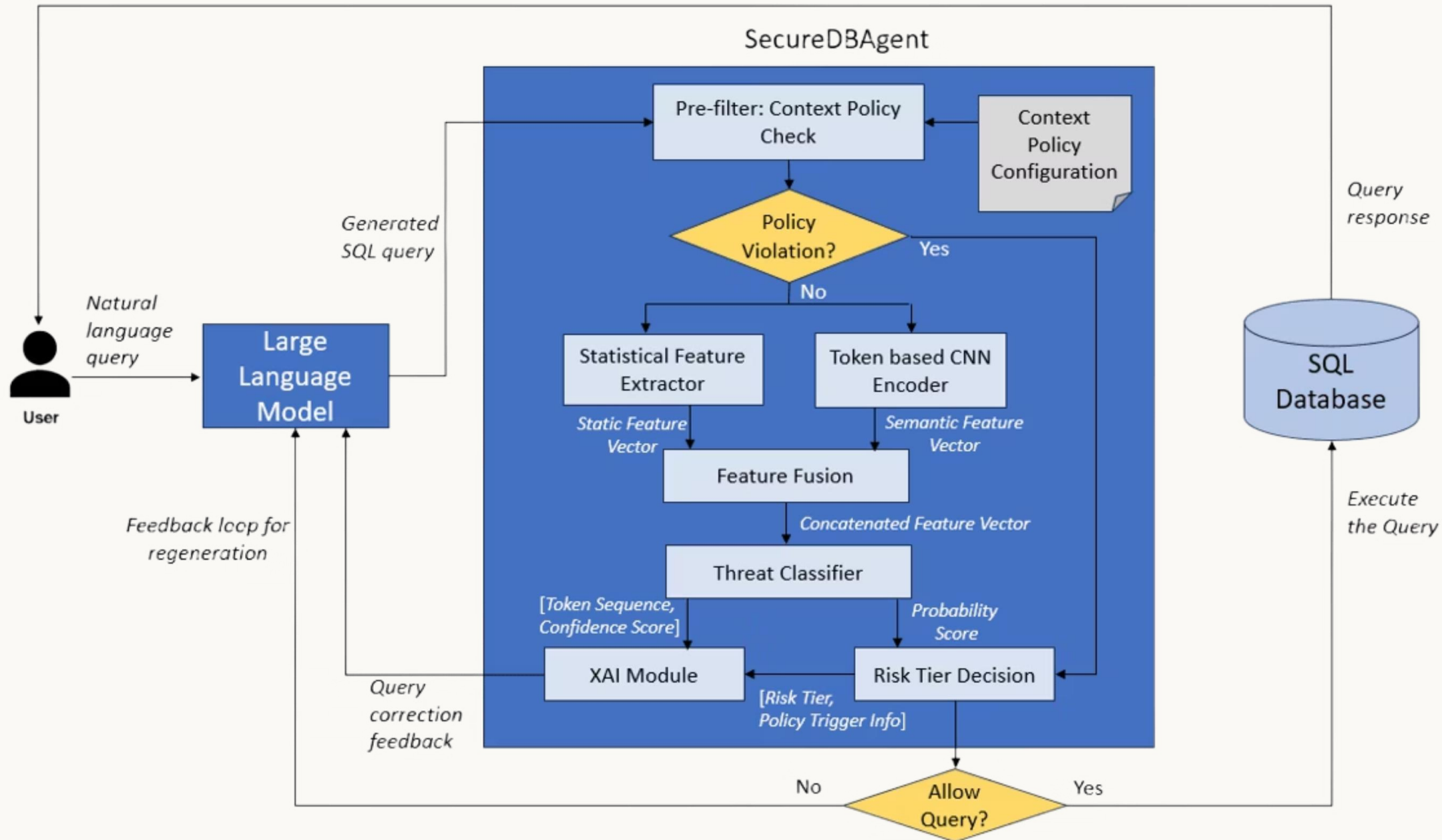
4

Rigorous Evaluation

Systematically evaluate the framework using standard benchmarks (SQLiV5, Kaggle, Spider) across key performance and security metrics.



SecureDB Agent – System Architecture



Context Policy Engine: Motivation and Role in the architecture



Why a Context-Aware Policy Layer?

- LLM-generated SQL can be syntactically correct but contextually malicious.
- Classical SQLi detection fails when queries violate role, history, or organisational norms.
- Existing solutions don't integrate behavioural context with semantic ML models effectively.



Role of the Context Policy Engine

- Serves as a pre-filter before the ML threat classifier.
- Performs deterministic policy enforcement and data-driven anomaly detection.
- Provides the first line of defence, producing ALLOW / FLAG / BLOCK decisions.



Novelty

- Introduces a two-layer fusion of handcrafted RBAC policy and learned behavioural detectors.
- Converts static database access governance into a dynamic, context-aware risk assessment pipeline.
- Ensures contextual violations can block queries, even if deemed low-risk by ML models alone.

Internal Architecture of the Context Policy Engine

The Context Policy Engine operates through a multi-layered approach, combining stringent deterministic policy checks with adaptive, data-driven behavioural detectors to ensure secure AI-to-database interaction.



1. Hard Deterministic Policy Checks

- Enforces role-based access (read/write/DDL).
- Prevents disallowed table-tag combinations (e.g., PII).
- Limits multi-statement & comment-based queries.
- Prohibits OS/server function calls for non-admins.
- Applies temporal constraints (working hours).



2. Data-Driven Behavioural Detectors

- **Plausibility Detector:** Scores query validity based on historical usage patterns (tables, joins, columns).
- **Provenance Novelty Detector:** Measures how unusual a query's table-tag pattern is compared to the user's history.
- **Session Anomaly Detector:** Evaluates session-level risks like query rate spikes or unexpected operations.
- **Schema Integrity Detector:** Identifies suspicious joins by assessing historical co-occurrence and table sensitivity.



3. Fusion & Decision Logic

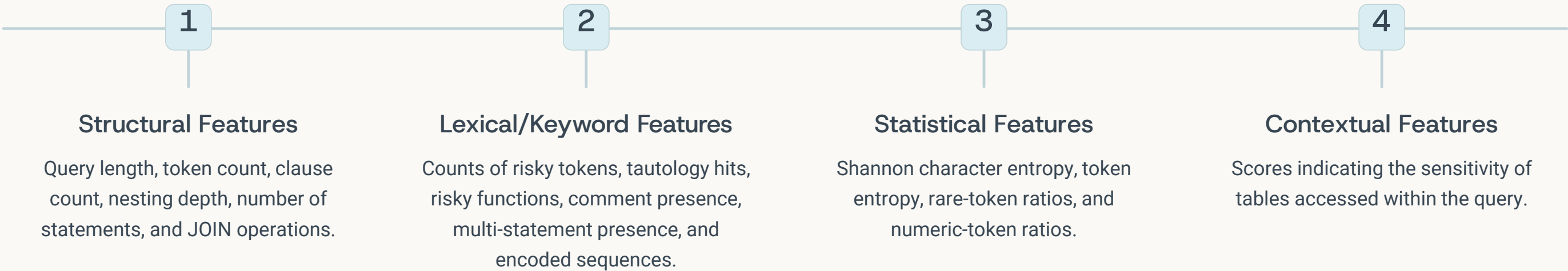
- Hard policy checks run first for immediate blocking. Data-driven detectors then assess the query.
- Decisions to BLOCK, FLAG, or ALLOW are based on a weighted fusion of plausibility, novelty, anomaly scores, and schema integrity findings.
- This transforms static RBAC into dynamic, probabilistic governance.

Static Feature Extractor

Why Static Features Are Needed

- Pure semantic encoders focus on meaning, but SQL attacks often exploit structural modifications.
- Attackers can alter operators, clause structures, comment placement, or rare-token patterns, which are often invisible to semantic analysis alone.
- Static features enable detection of non-semantic anomalies like tautology injections, high-entropy obfuscation, and multi-statement exploits.

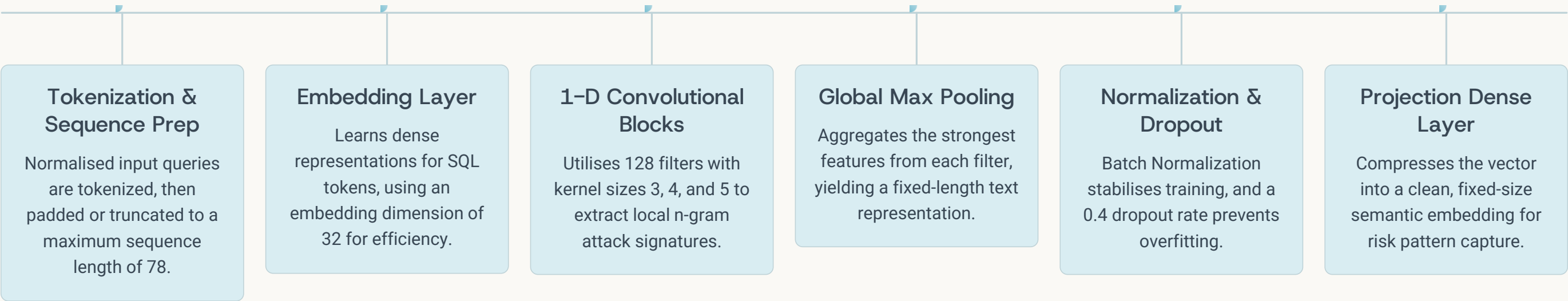
Feature Categories Extracted



The static feature extractor transforms SQL queries into a behavioural fingerprint, allowing hybrid models to detect structural anomalies that semantic encoders systematically miss. Unlike prior work (SQLiV3–V5, AutoSQLi, GPT-SQLi papers), this thesis tightly integrates handcrafted statistical features and learned embeddings, enabling robust detection of LLM-generated malicious SQL that is semantically fluent but structurally anomalous.

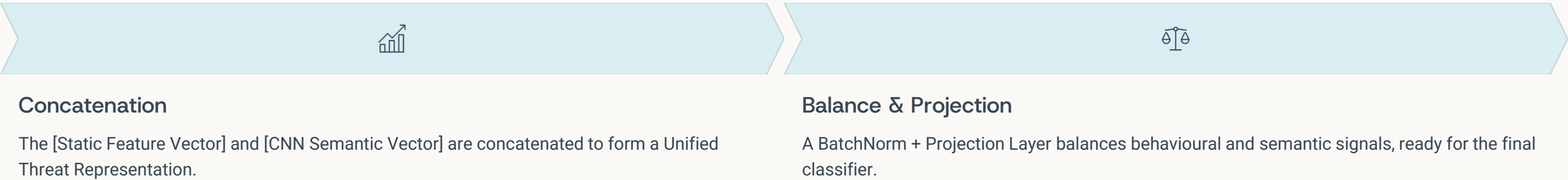
CNN-Based Semantic Encoder

This component transforms raw SQL queries into a rich semantic embedding, capable of detecting complex attack patterns.



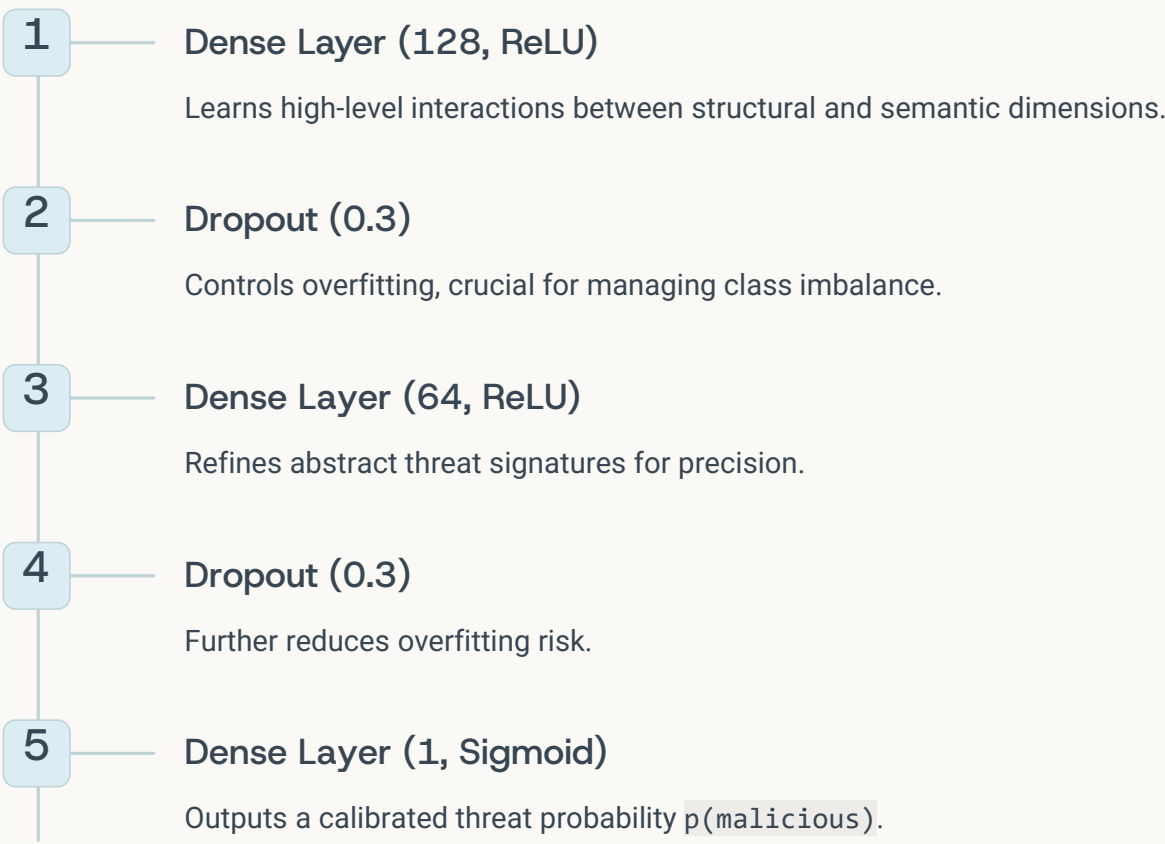
Feature Fusion Layer

This layer brings together disparate signals to create a comprehensive threat profile for the SQL query.



Threat Classifier – MLP for Unified Decision Making

The architecture processes a fused feature vector, combining semantic embeddings with static features.



Training Setup: Utilises binary cross-entropy loss with an Adam optimizer and early stopping (patience = 7 epochs) for robust learning.

This classifier uniquely learns joint interactions between textual SQL and engineered features, surpassing traditional methods. Our ablation studies confirm that this fusion architecture significantly outperforms CNN-only or static-only models in F1 and ROC-AUC scores, a synergy not previously applied to LLM-generated SQL threat modelling.

Risk Tier Decision

While a simple probability score of malicious intent ($p(\text{malicious})$) is valuable, it's insufficient on its own. Different operational contexts demand tailored responses. Thresholds convert this raw probability into actionable decisions: **ALLOW**, **FLAG**, or **BLOCK**.

Determining Optimal Thresholds

To establish robust decision boundaries, we compute two critical curves on our validation dataset:

ROC Curve (TPR vs FPR)

Used to find Youden's Optimal Threshold:

$$Youden = \arg \max_t (TPR(t) - FPR(t))$$

This method meticulously balances the likelihood of accurately catching malicious queries while stringently minimising false alarms.

PR Curve (Precision vs Recall)

Used to identify the Max-F1 Threshold:

$$t = \arg \max_t F1(t)$$

This approach specifically optimises the delicate trade-off between the risk of over-blocking legitimate queries and inadvertently missing genuine attacks.

In our system, both curves impressively converged, yielding an optimal threshold of approximately **0.78**. This ensures a high recall rate for malicious queries, concurrently maintaining an extremely low False Positive Rate (FPR) on our validation data.

Risk Tiers

LOW

Threshold: $p < 0.40$; Action: **Allow**

The query is highly likely to be benign and poses minimal threat.

MEDIUM

Threshold: $0.40 \leq p < 0.70$; Action: **Flag**

The query exhibits suspicious signals, requiring further human review or additional context checks by the system.

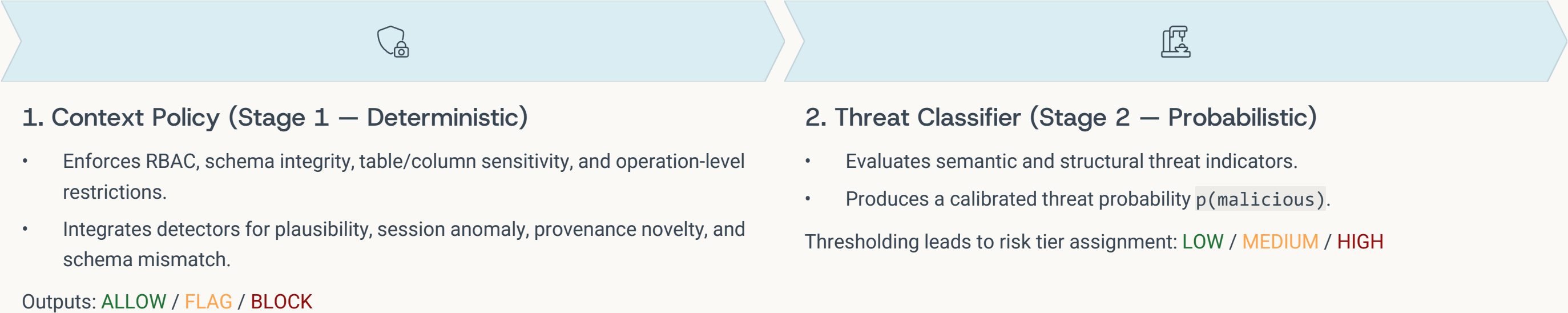
HIGH

Threshold: $p \geq 0.70$; Action: **Block**

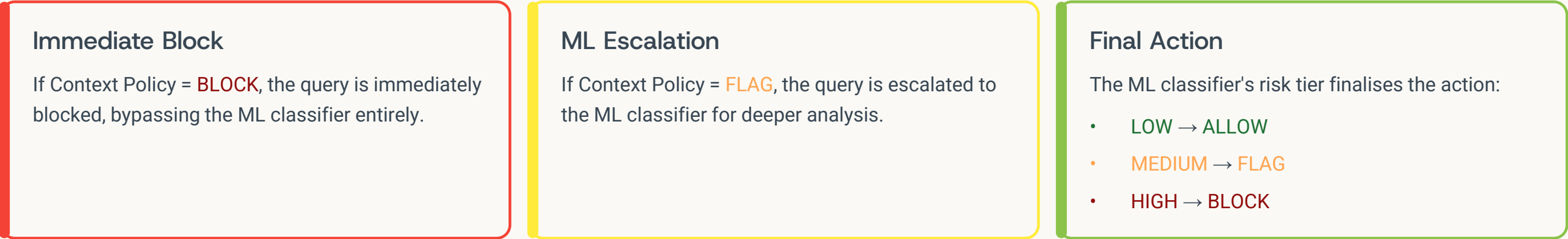
There is a high confidence that the query is malicious, structurally anomalous, or violates critical policies.

End-to-End Secure DB Agent Pipeline

A SQL query undergoes a two-stage defence pipeline, merging rule-based guarantees with ML-driven detection:



Joint Decision Logic



Explainable AI Engine

Database security demands traceability, requiring admins and auditors to understand precisely why a SQL query was flagged or blocked.

Traceability is Key

Admins and auditors need clear explanations for all flag/block decisions to ensure system integrity and compliance.

SHAP Limitations

Standard SHAP (SHapley Additive exPlanations) struggles with our hybrid model's mixed input (CNN + static features).

Hybrid XAI Solution

Our custom Explainable AI (XAI) design overcomes these challenges with a unique hybrid explainer architecture.

Three-Component XAI Framework

1

Static Feature SHAP Explainer

Utilises KernelSHAP on the static-only projection model to quantify contributions of statistical features, such as:

- Tautology scores
- Rare token ratios
- Comment presence
- Entropy measures

This allows precise statements like: "High tautology-hit count contributed **+0.35** to malicious probability."

2

Token-Level Semantic Saliency

For CNN embeddings where SHAP is less effective, we employ an occlusion-based approach:

- Remove/mask one token at a time
- Observe the change in $p(\text{malicious})$

This generates a ranked list of influential tokens (e.g., "UNION", "SLEEP()", "--") visualised as heatmaps or saliency bars.

3

Context-Policy Reasoning Trace

Provides a clear trace of which specific rule or detector within the Context Policy Engine triggered a decision:

- "Role violates permission: appuser cannot access financial tables"
- "Session rate anomaly: spike **+23x** baseline"
- "Table 'salary' tagged 'confidential': read access blocked"



Our system uniquely offers a unified XAI framework that seamlessly explains model-based, rule-based, and behavioural detectors in a single, coherent narrative. This comprehensive approach ensures each block/flag decision is transparent and accountable, aligning with human-centered AI principles.

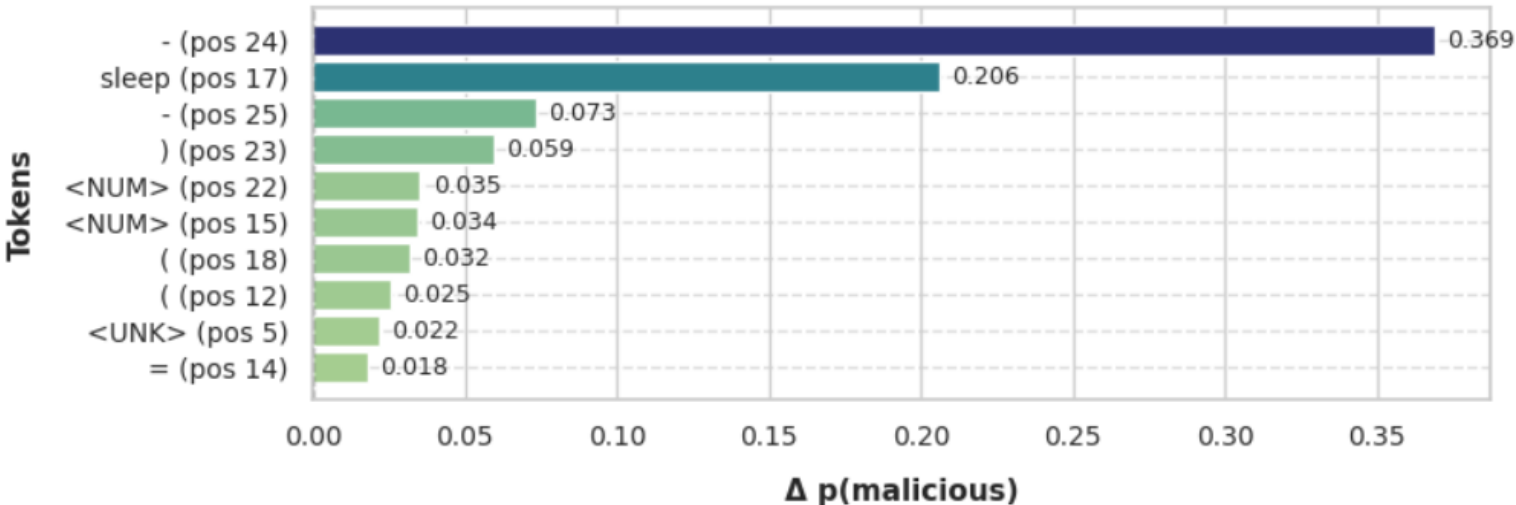
Output format with Explainable AI

SecureDBAgent XAI Dashboard

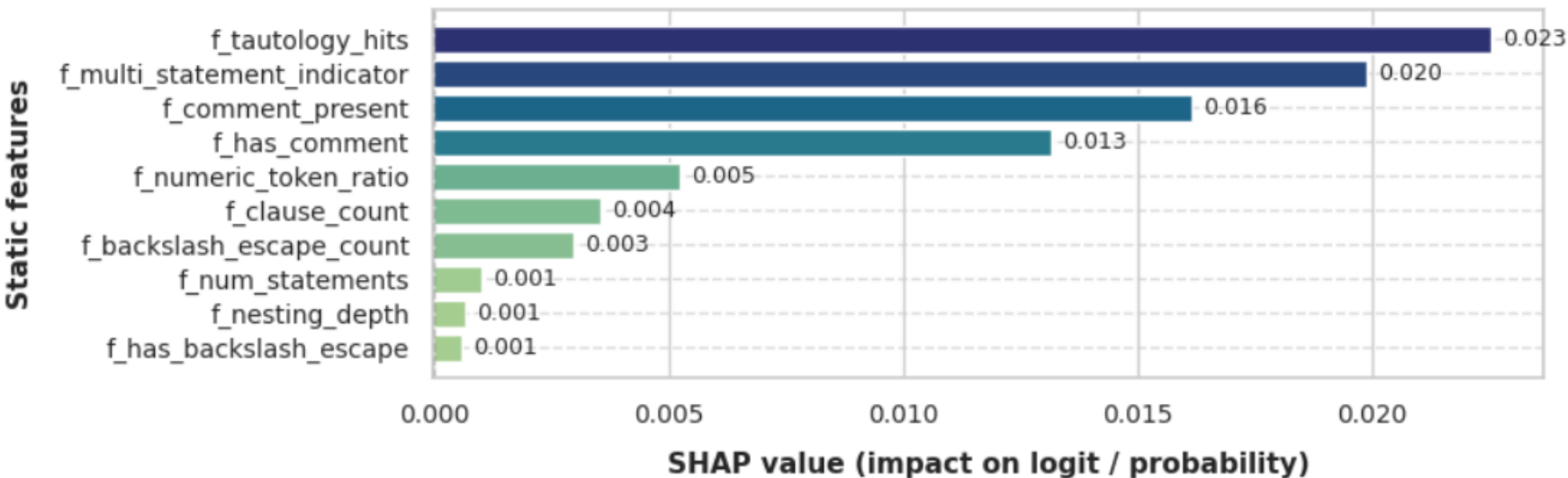
Query: `SELECT * FROM employee WHERE user_id = '1';IF(1=1, SLEEP(5), 0)--';`

Field	Value
Final action	BLOCK
Risk tier	HIGH
Threat probability (fusion model)	0.989
Context policy decision	FLAG
Context reasons	- unauthorized_read_for_tags

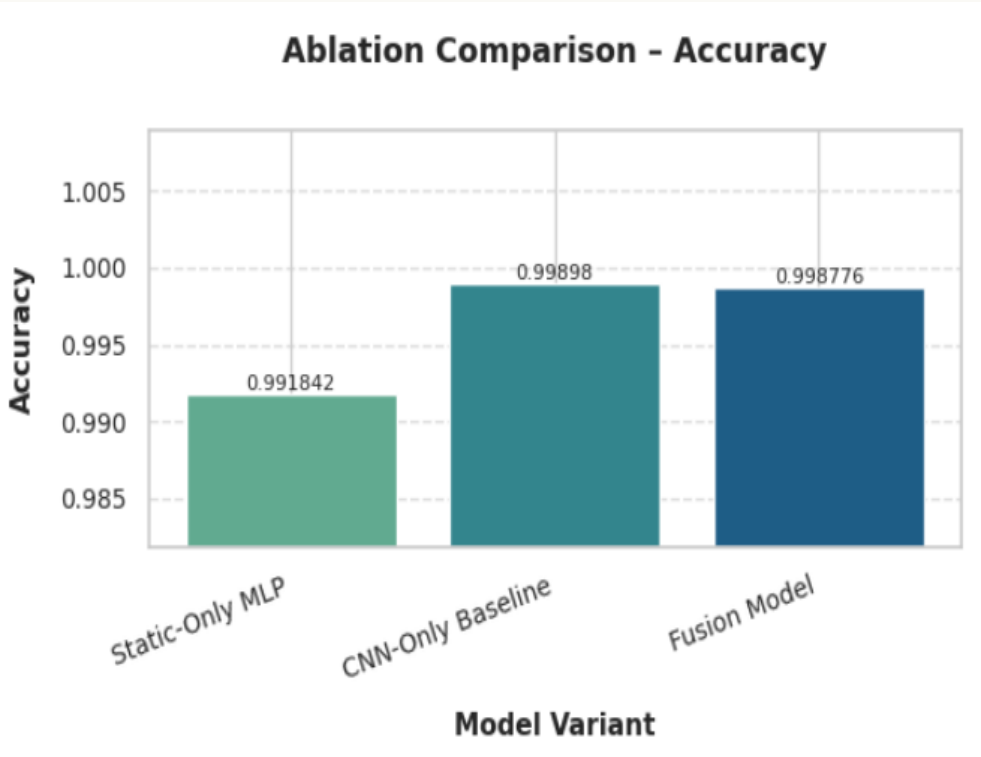
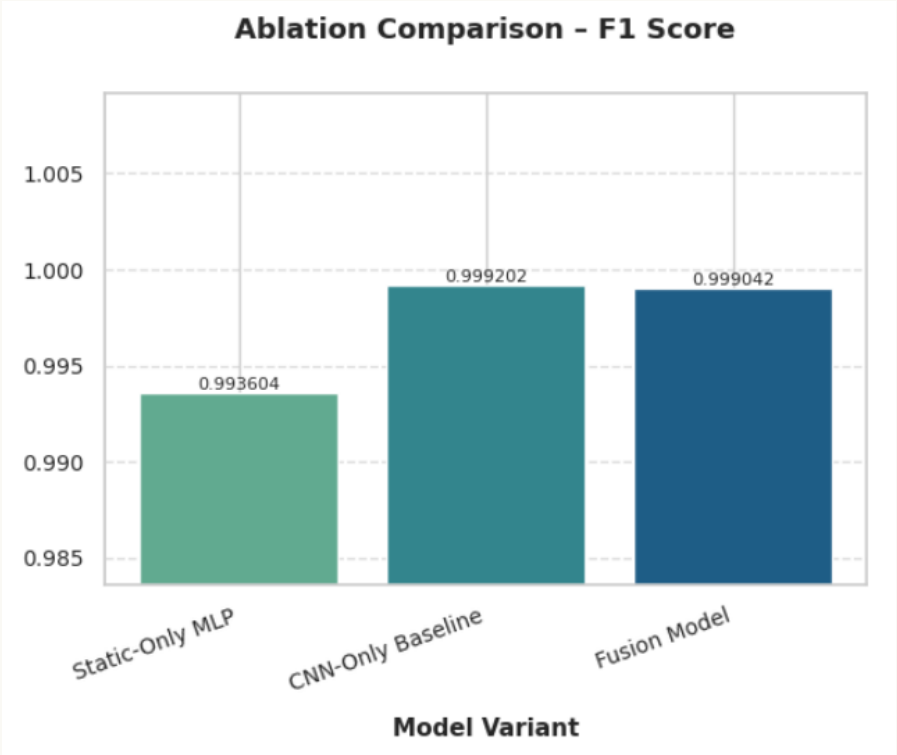
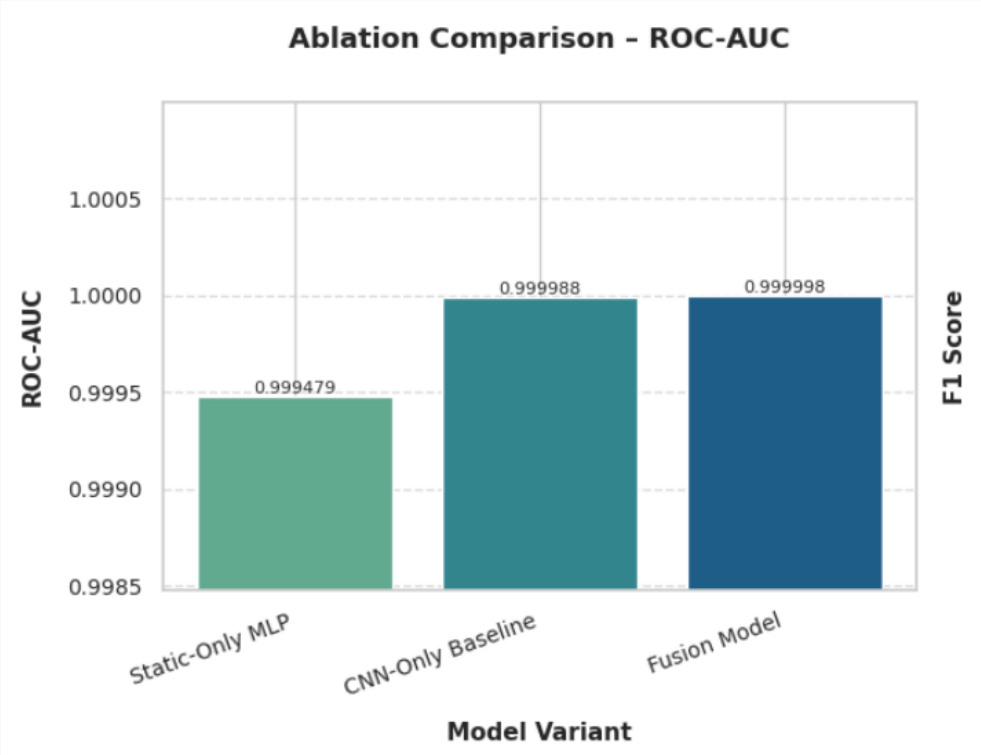
Token-level saliency (Δ probability when masked)



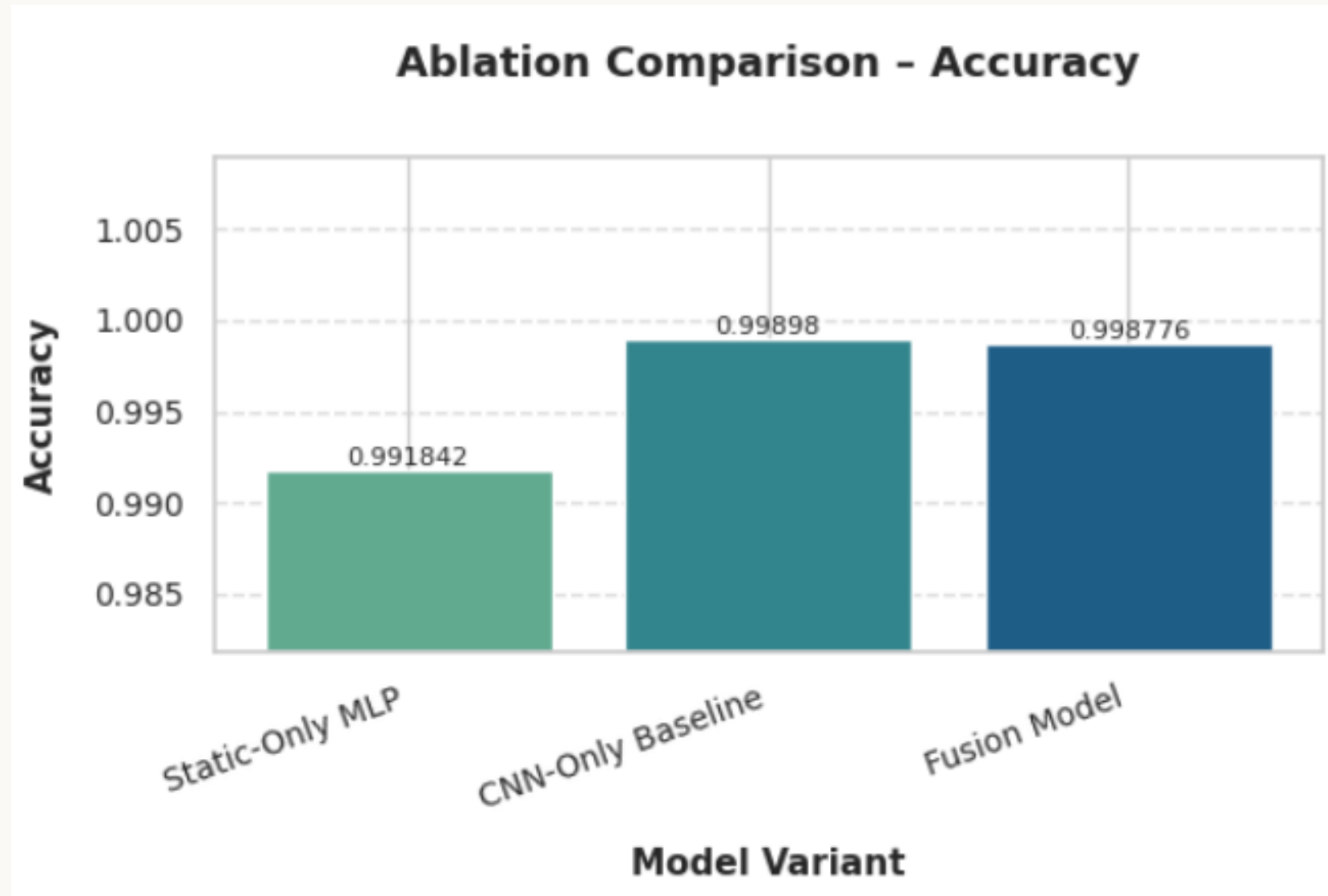
Local static feature explanation (SHAP)



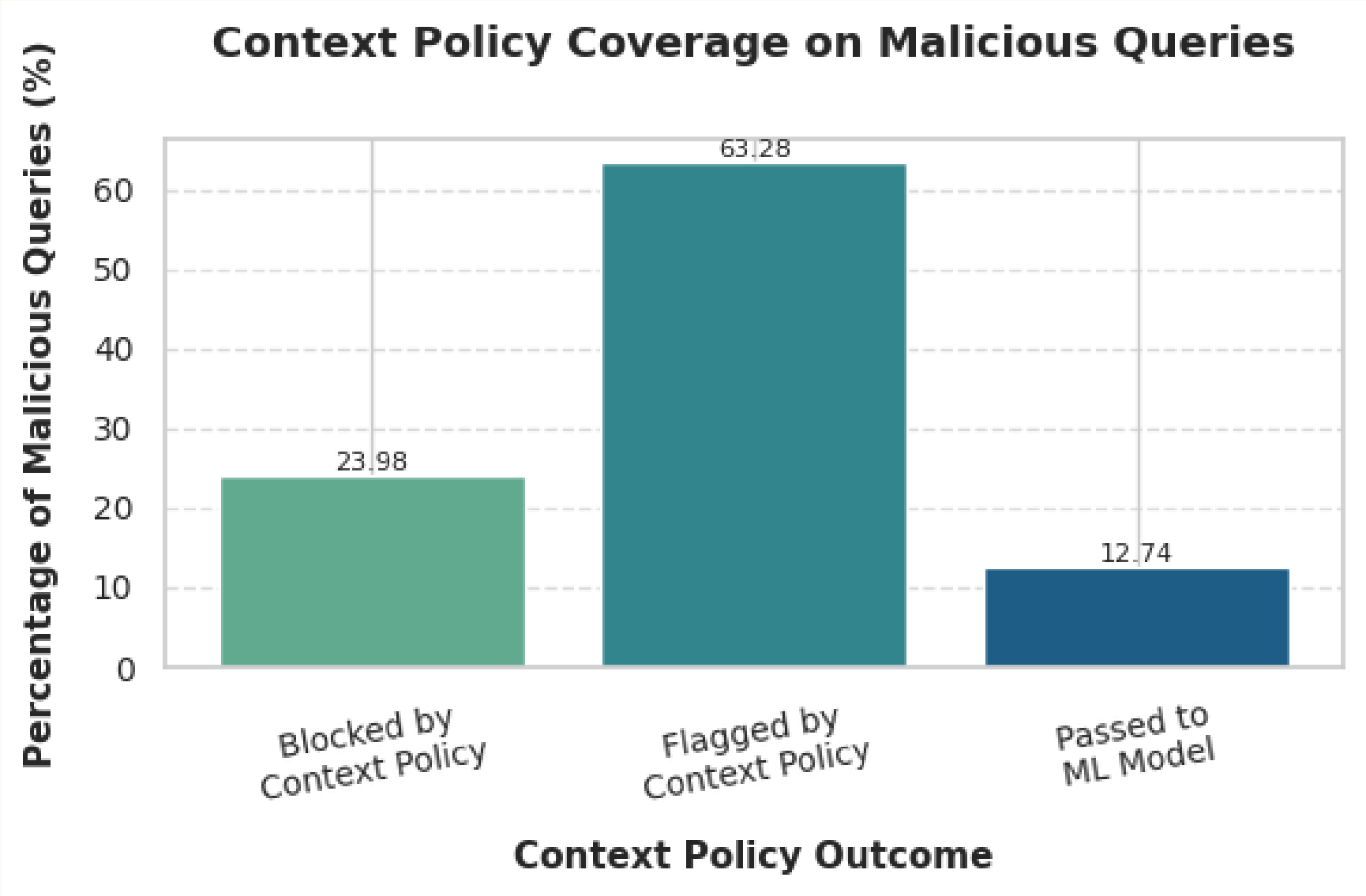
Ablation Study Results – Threat Classifier



Spider Dataset – Generalization



Context policy coverage on Malicious queries



Full Pipeline results – Context policy+Fusion model

	roc_auc	pr_auc	precision	recall	f1	accuracy
0	0.999198	0.999504	0.999723	0.999169	0.999446	0.999184

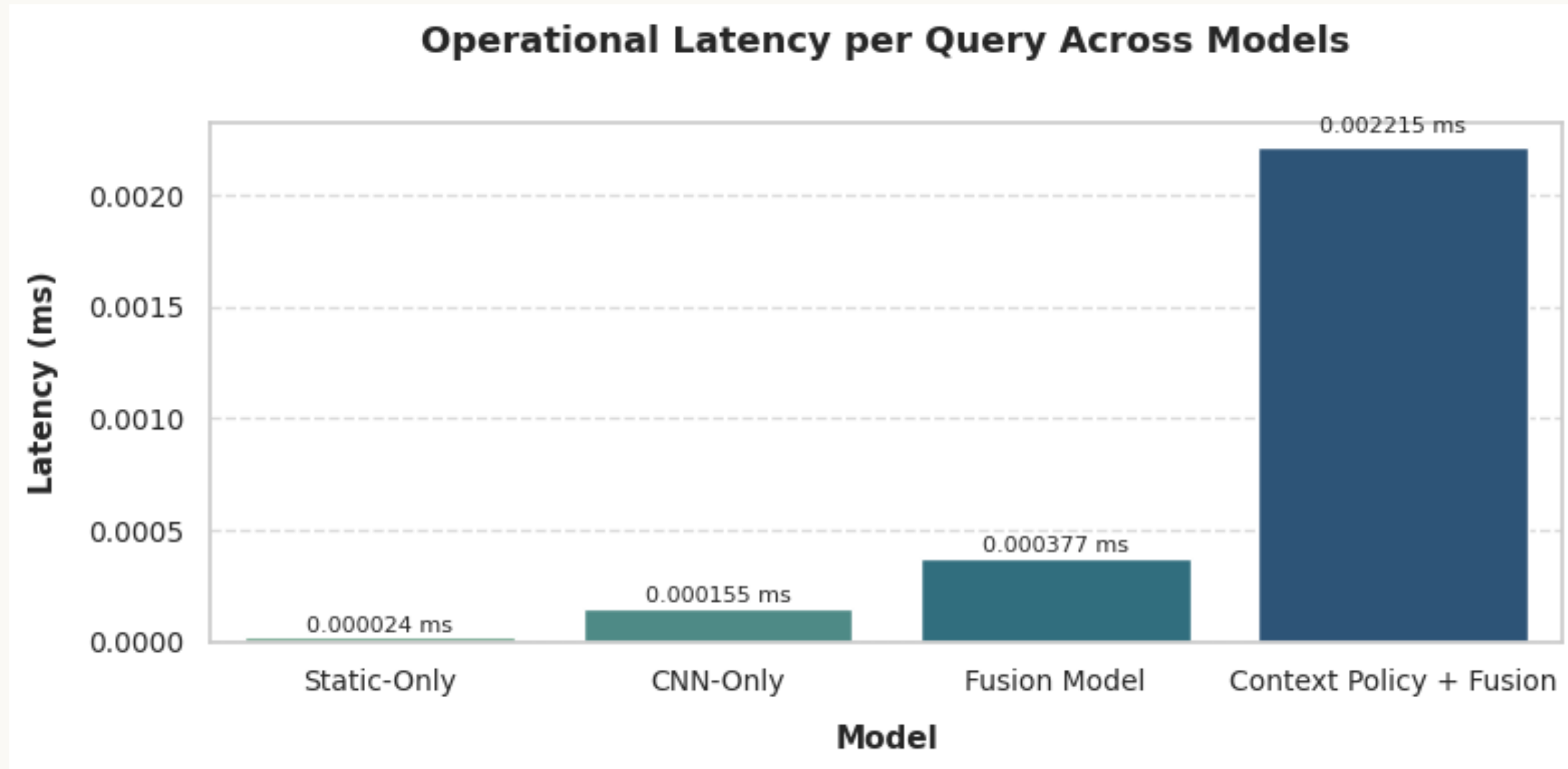
=== Confusion Matrix (rows = actual, cols = predicted) ===

	Pred 0 (benign)	Pred 1 (malicious)
Actual 0 (benign)	1291	1
Actual 1 (malicious)	3	3608

Breakdown:

- TN (benign correctly allowed) : 1291
- FP (benign incorrectly flagged) : 1
- FN (missed attacks) : 3
- TP (attacks correctly caught) : 3608

Operating latency



Future Directions & Research Opportunities

Our journey towards robust and intelligent database security continues. Here are key areas for future development and research:



Fine-grained Semantic Role Modelling

Integrate graph-based provenance or Knowledge Graph embeddings to capture nuanced user–data interaction patterns over time for richer contextual understanding.



Dynamic Adaptive Policies

Transition from static Role-Based Access Control (RBAC) configurations to self-learning policies that intelligently evolve with organizational usage patterns.



Cross-Query Behavioural Graphing

Develop methods for linking sessions across diverse applications to detect complex, correlated anomalies that span multiple interaction points.



Unified Tri-Modal Explainer

Combine SHAP, token saliency, and rule-trace reasoning into a single, cohesive visual narrative for comprehensive explainability.



Natural-Language Rationales

Implement auto-generation of clear, admin-facing explanations to streamline audit workflows and enhance operational transparency.



Real-time Streaming Integration

Seamlessly integrate with production database proxies like PgBouncer or MySQL Proxy for real-time threat detection and mitigation.



Edge Deployments

Explore deployments for privacy-sensitive environments utilising quantized or distilled versions of our fusion model for efficiency and security.

Conclusion

Core Contributions

- Designed and implemented a multi-layer Secure DB Agent combining deterministic context-policy enforcement with a deep-learning fusion model.
- Achieved state-of-the-art detection performance, especially against LLM-generated SQL injection patterns.
- Developed a novel hybrid XAI framework that explains decisions at static feature level, token level, and policy-rule level.

Novelty in Proposed Approach

- Moves beyond traditional SQLi detection by integrating role semantics, session behavior, schema sensitivity, and ML inference.
- Introduces contextually-aware decision boundaries, invoking ML selectively based on policy signals.
- Demonstrates that explainability can be operational with interpretable traces for every block or flag.

Impact & Implications

- Provides a robust, explainable, and enterprise-ready blueprint for securing databases from AI-generated threats.
- Establishes a framework for future academic research on multimodal security models and trustable AI in sensitive systems.
- Reinforces the importance of hybrid intelligence—combining rule-based guarantees with learned representations.

This thesis demonstrates that Secure AI is not a choice between rules and learning – it is the intelligent blending of both.

